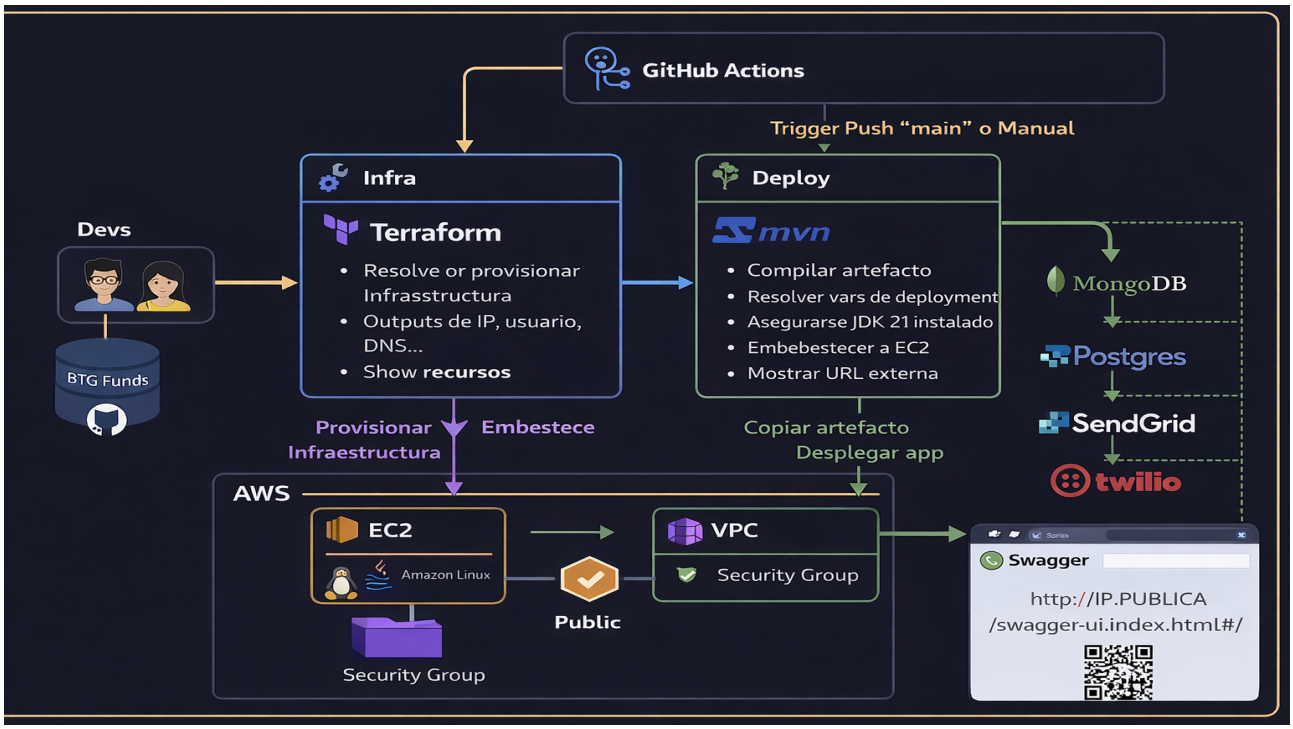
Arquitectura Técnica de BTG Funds

CI/CD con GitHub Actions, aprovisionamiento con Terraform, despliegue en AWS EC2 y arquitectura n-capas con Spring Boot, JPA, MongoDB y JWT.

Documento técnico de soporte para explicar las decisiones arquitectónicas, la automatización del despliegue y la estructura interna de la aplicación.



1. Visión general

La solución BTG Funds fue diseñada para cubrir dos objetivos técnicos principales: primero, exponer una API backend segura y mantenible para la gestión de fondos, clientes, suscripciones, transacciones y notificaciones; segundo, permitir que la infraestructura y el despliegue de la aplicación sean automáticos, repetibles y auditables. Para lograrlo, se definió una arquitectura compuesta por dos dimensiones complementarias: una **arquitectura de despliegue** basada en GitHub Actions, Terraform y AWS; y una **arquitectura interna n-capas** en Spring Boot, separando responsabilidades por configuración, seguridad, controladores, servicios, repositorios y modelos de datos.

2. Arquitectura de despliegue

El pipeline CI/CD se ejecuta desde GitHub Actions y se divide en dos responsabilidades lógicas: **Infra** y **Deploy**. Aunque ambas viven dentro del mismo workflow, están separadas por jobs para conservar claridad técnica y permitir dependencias explícitas entre etapas.

- **Infraestructura con Terraform:** resuelve si la instancia EC2 ya existe o si debe aprovisionarse, evitando recreaciones innecesarias.
- **Despliegue de aplicación:** compila el artefacto JAR con Maven, resuelve variables de conexión, valida Java en la máquina remota y despliega la aplicación en EC2 mediante SSH/SCP.
- **Exposición pública:** la aplicación queda disponible a través de la IP o DNS público de la instancia, incluyendo la interfaz Swagger para documentación y pruebas.

2.1 Por qué se usó Terraform

Terraform se eligió porque permite definir la infraestructura como código. Esto implica que la instancia EC2, el Security Group, las variables y los outputs quedan versionados junto al proyecto. Técnicamente esto aporta repetibilidad, trazabilidad de cambios, facilidad para reconstruir ambientes y reducción del error manual. También permite expresar dependencias explícitas entre recursos, por ejemplo entre una instancia EC2 y el Security Group que la protege.

2.2 Por qué se usó GitHub Actions

GitHub Actions se utilizó como motor CI/CD porque se integra directamente con el repositorio, permite ejecutar pruebas, compilar, aprovisionar y desplegar sin necesidad de un servidor adicional, y facilita la gestión de secretos como credenciales AWS, URI de MongoDB, credenciales de PostgreSQL, SendGrid, Twilio y variables de verificación del login.

2.3 Flujo técnico del pipeline

1. El desarrollador realiza un push a la rama principal o ejecuta el workflow manualmente.
2. El job de infraestructura valida si la EC2 existe; si no existe, Terraform la crea.
3. Se resuelven los datos públicos de la máquina desde AWS, como IP y DNS público.
4. El job de despliegue ejecuta pruebas, empaqueta el proyecto y transfiere el JAR a EC2.
5. Se genera el archivo application.properties con variables sensibles inyectadas desde GitHub Secrets.
6. Se crea o actualiza un servicio systemd para que la aplicación se ejecute como servicio del sistema.
7. Finalmente, se verifica el funcionamiento mediante un endpoint real de login y la interfaz Swagger.

3. Arquitectura n-capas de la aplicación

La estructura interna del proyecto sigue un enfoque de **arquitectura n-capas**. Esto significa que el sistema se organiza por responsabilidades técnicas y no como una sola unidad acoplada. En el proyecto se observa claramente en paquetes como *config*, *controller*, *dto*, *entity*, *repository*, *service*, *security*, *exception* y *notification*.

- **Capa de presentación (controller):** expone los endpoints REST y recibe las solicitudes HTTP.
- **Capa de aplicación/negocio (service):** contiene la lógica funcional del sistema, reglas de negocio, validaciones y orquestación.
- **Capa de acceso a datos (repository):** encapsula la persistencia contra MongoDB o PostgreSQL.
- **Capa de modelos y transferencia (entity, document, dto):** separa modelos persistentes de objetos usados para entrada y salida de la API.
- **Capa transversal (security, config, exception, notification):** centraliza seguridad, configuración, errores y notificaciones.

3.1 Por qué se definió arquitectura n-capas

Técnicamente, esta arquitectura se definió para mejorar la mantenibilidad del backend. Separar controladores, servicios y repositorios evita mezclar lógica HTTP con reglas de negocio o acceso a base de datos. Esto facilita pruebas unitarias, escalabilidad del código, reemplazo de implementaciones y mejor legibilidad para otros desarrolladores o evaluadores técnicos. Además, permite que cambios en una capa tengan menor impacto sobre las demás.

4. Persistencia híbrida: MongoDB + PostgreSQL

El proyecto utiliza persistencia híbrida porque resuelve necesidades distintas con tecnologías distintas. No se eligió una sola base de datos para todo, sino la combinación más adecuada según el tipo de información.

4.1 MongoDB

MongoDB Atlas se utiliza para la parte principal del negocio de fondos. Es especialmente útil para manejar documentos de clientes, fondos, suscripciones, transacciones y notificaciones, donde la flexibilidad del esquema ayuda a iterar más rápido y a almacenar estructuras documentales sin el costo de cambios relacionales frecuentes.

4.2 PostgreSQL con JPA

PostgreSQL se usa para la parte SQL del ejercicio y para escenarios donde el modelo relacional es más natural. Sobre PostgreSQL se emplea **JPA** para mapear entidades Java a tablas relacionales. Esto aporta una forma estándar y productiva de persistencia orientada a objetos, facilita consultas, relaciones y separación entre entidades y repositorios, y encaja bien en una arquitectura empresarial basada en Spring Boot.

4.3 Por qué se usó JPA

JPA se eligió porque reduce la complejidad del acceso relacional comparado con SQL manual para las operaciones básicas. Permite trabajar con entidades, repositorios y relaciones con menos código repetitivo, manteniendo una estructura clara en el proyecto. Técnicamente mejora productividad, consistencia y facilidad de mantenimiento.

5. Seguridad con JWT y roles

La API implementa seguridad basada en **JWT** y autorización por **roles**. Esta decisión responde a la necesidad de exponer endpoints REST seguros y desacoplados del estado de sesión.

5.1 Por qué se usó JWT

JWT se eligió porque es adecuado para APIs stateless. Después de autenticarse, el usuario recibe un token firmado que puede enviar en cada request. Esto evita mantener sesiones en servidor, simplifica la escalabilidad horizontal y es una práctica común en backends modernos. Además, se integra bien con filtros de seguridad y autorización por rutas.

5.2 Roles ADMIN y CLIENT

Se definieron los roles **ADMIN** y **CLIENT** porque el dominio del problema exige distintos niveles de acceso. Un administrador debe poder crear, revisar o gestionar información con privilegios elevados; mientras que un cliente debe consumir únicamente las operaciones que le corresponden, como consultar fondos disponibles, suscribirse, cancelar suscripciones o revisar su historial. Desde el punto de vista técnico, esta separación minimiza exposición indebida de funcionalidades y aplica el principio de mínimo privilegio.

- **ADMIN:** orientado a operaciones administrativas o de gestión global del sistema.
- **CLIENT:** orientado a acciones de negocio del usuario final dentro de sus propios límites.

5.3 Componentes de seguridad

La seguridad se apoya en clases dedicadas para generación y validación del token, filtros de autenticación, configuración de seguridad, definición de rutas protegidas y extracción del rol del usuario autenticado. Esto evita acoplar la lógica de seguridad a los controladores y mantiene la solución alineada con buenas prácticas de Spring Security.

6. Integraciones externas

Además de AWS y las bases de datos, la aplicación depende de servicios externos de comunicación y almacenamiento.

- **MongoDB Atlas:** persistencia documental del negocio principal.
- **PostgreSQL:** persistencia relacional para la parte SQL y modelos estructurados.
- **SendGrid:** envío de notificaciones por correo electrónico.
- **Twilio:** envío de notificaciones por SMS.

7. Exposición del servicio y observabilidad básica

Una vez desplegada, la API queda publicada en una instancia EC2 con Security Group configurado para permitir el tráfico necesario. El servicio se ejecuta mediante **systemd**, lo que permite controlarlo como daemon del sistema, reiniciarlo automáticamente y revisar sus logs con journalctl. Adicionalmente, la interfaz Swagger se expone para probar los endpoints y evidenciar funcionalidad del backend de forma visual.

8. Justificación técnica de la solución completa

- Terraform aporta infraestructura reproducible y controlada por código.

- GitHub Actions automatiza validación, construcción y despliegue sin servidor adicional.
- AWS EC2 ofrece una plataforma simple y suficiente para alojar el backend.
- systemd asegura ejecución estable de la aplicación como servicio del sistema.
- Spring Boot facilita una base empresarial robusta para exponer la API.
- La arquitectura n-capas mejora mantenibilidad, escalabilidad y claridad.
- MongoDB y PostgreSQL cubren correctamente necesidades documentales y relacionales.
- JWT con roles ADMIN y CLIENT fortalece la seguridad y el control de acceso.
- SendGrid y Twilio cubren canales de notificación reales del negocio.

En conjunto, esta arquitectura no solo resuelve el despliegue técnico del proyecto, sino que también demuestra decisiones de ingeniería justificadas: separación de responsabilidades, automatización, seguridad, persistencia adecuada al problema y capacidad de evolución.